



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

INF3500 - Conception et réalisation de systèmes numériques

Laboratoire 2

Marc-Antoine MANNINGHAM
Beaurel FOHOM

Automne 2025

Feu de circulation

À la fin de ce laboratoire, vous devrez être capable de :

- Concevoir un module séquentiel pour qu'il rencontre ses spécifications, donner un diagramme d'états et son implémentation résultante en VHDL.
- Concevoir un banc d'essai pour stimuler le modèle VHDL d'un circuit séquentiel, générer et analyser le chronogramme résultant de la simulation
 - Utiliser des types abstraits
 - Générer des stimuli pour un circuit séquentiel
 - Utiliser des énoncés `assert` ou des conditions pour vérifier le module.
- Implémenter un circuit séquentiel sur un FPGA et en vérifier le fonctionnement correct
 - Utiliser des interfaces comme des boutons et des affichages à LED.
 - Constater et corriger les phénomènes de rebond des boutons et commutateurs, traiter les appuis simultanés.

Ce laboratoire s'appuie sur et complète le matériel suivant :

1. Les concepts couverts dans les laboratoires 0 et 1
2. La matière du cours des thèmes 3 (Modélisation et vérification de circuits combinatoires) thème 4 (Modélisation et vérification de circuits combinatoires).

Liste des fichiers

Fichier	Contenu
src/feu.vhd	Module à compléter pour implémenter votre machine à états
src/monopulseur.vhd	Module monopulseur (<i>debouncer</i> en anglais) qui sert à supprimer les effets de rebond des boutons. Pour plus de détails, vous pouvez regarder cette vidéo
src/top.vhd	<i>Top level</i> du projet
src/feu_tb.vhd	Banc d'essai en VHDL à compléter pour tester votre machine à états en VHDL.
xdc/PYNQ-Z2-v1.0.xdc	Fichier de contraintes pour la PYNQ-Z2
synth-impl/create_zynq_wrapper.tcl synth-impl/zynq.tcl	Scripts de génération du <i>wrapper</i> pour le Zynq, à sourcer après avoir importé les sources

Introduction

Votre objectif dans ce laboratoire est de concevoir une machine à états (de Moore) pour simuler le fonctionnement d'un feu de circulation. À partir d'une spécification donnée, vous devrez concevoir une machine à états, donner un diagramme d'états, puis l'implémenter en VHDL, et enfin vérifier son fonctionnement en la programmant sur votre PYNQ.

La partie 0 présente le module déjà implémenté qui vous servira de base dans la suite du laboratoire. Dans la partie 1, vous devrez concevoir une machine à états qui respecte une spécification donnée, que vous implementerez et simulerez dans la partie 2. La PYNQ sera programmée pour vérifier le bon fonctionnement de votre machine dans la partie 3, et le défi dans la partie 4 consiste à implémenter une fonctionnalité supplémentaire à votre machine.

Partie 0 : Une machine à états basique

Exemple : des couleurs qui défilent

Le fichier `feu.vhd` contient un exemple de machine à états basique, implémente le diagramme d'états présenté dans la figure 1.

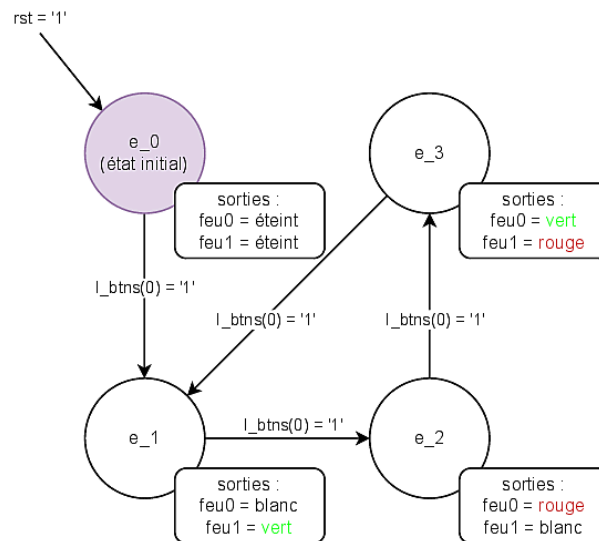


Fig. 1. – Machine à états basique

Cette machine commence dans l'état `e_0`, dans lequel les deux LEDs sont éteintes. C'est son état de réinitialisation. Ensuite, chaque appui du bouton 0 fait passer la machine à l'état suivant, ce qui a pour effet de faire défiler les couleurs vert, blanc et rouge sur les deux LEDs RGB. L'état `e_3` reboucle sur l'état `e_1` ; le seul moyen d'atteindre l'état `e_0` est de réinitialiser la machine en utilisant le **reset**, relié au bouton 1.

Les quatre états de cette machine sont déclarés dans l'architecture du module `feu` par :

```
type etat_type is (e_1, e_2, e_3, e_0);
```

L'état du système est porté dans le module par le signal `etat` :

```
signal etat : etat_type := e_0;
```

Info

Le mot-clé `type` déclare un type abstrait, un objet VHDL qui permet de manipuler des valeurs sans se soucier de leur valeur numérique (semblable aux `enum` en C). Le signal `etat` sera synthétisé comme le nombre nécessaire de registres pour pouvoir encoder toutes les valeurs possibles de `etat`, déclarées par `etat_type` ; à la manière d'un `enum`, l'état `e_1` est encodé par "00", `e_2` par "01", `e_3` par "10" et `e_0` par "11". Cela correspond à l'ordre dans lequel ils sont déclarés.

Cette machine simple est implémentée dans `feu.vhd` en guise d'exemple sur lequel vous appuyer pour développer la machine à états demandée dans la partie 1.

Avant de passer à la partie suivante et à l'implémentation de votre propre machine selon la spécification requise, vous allez simuler et implémenter le module de démonstration.

0a. Simulation

Pour commencer, vous allez simuler le module. Créez un projet Vivado, ajoutez-y vos sources (reportez-vous au tutoriel du laboratoire 0 si besoin), puis définissez `feu_tb.vhd` comme *top level* de la simulation, et lancez la simulation.

Vous devriez constater que le banc d'essai stimule le bouton d'entrée un tick d'horloge sur deux. Cela permet de tester à la fois les transitions, et la persistance des états. Vous pouvez aussi observer que les deux sorties du module suivent un cycle de valeurs en décalé. Prenez le temps de **lire le code source banc d'essai et du module** et les commentaires pour bien les comprendre.

Évaluation

Ajoutez une capture d'écran du résultat de votre simulation à votre rapport.

0b. Implémentation

Une fois le module simulé et le banc d'essai compris, définissez `top.vhd` comme *top level* de la synthèse (ça devrait être le cas par défaut), puis synthétisez, implémentez et compilez

le design pour obtenir un *bitstream*. Programmez votre carte pour vérifier le bon fonctionnement du module.

Tiens tiens tiens... vous devriez constater que le module ne se comporte pas comme prévu. Pourtant, la simulation indique que les leds passent bien de « vert / blanc » à « blanc / rouge » au premier appui du bouton, « blanc / rouge » à « rouge / vert » au second, « rouge / vert » à « vert / blanc » au troisième, et ainsi de suite.

! Évaluation

Comment se fait-il que l'implémentation sur la carte ne semble pas fonctionnelle ? Écrivez quelques phrases dans votre rapport pour **expliquer ce phénomène**, et **implémentez un process en VHDL pour y remédier**. Vous pouvez ajouter autant de signaux intermédiaires que vous le souhaitez. *Veillez à bien commenter votre code pour expliquer votre démarche.*

Peut-on réellement dire que la machine implémentée est dysfonctionnelle ?

! Attention

Vous ne devez pas utiliser le module `monopulseur` implémenté dans `monopulseur.vhd`. Nous vous demandons d'**implémenter votre propre process séparé des autres**, que vous réutiliserez pour les parties suivantes.

Cette partie devrait être finie lors de la séance de laboratoire, afin que vous puissiez mener à bien le reste du sujet par vous-mêmes. **Assurez-vous de l'avoir bien comprise avant de passer à la suite.**

Partie 1 : Feu de circulation

Spécification

Dans cette partie, il vous est demandé d'implémenter une machine à états qui respecte la spécification suivante pour implémenter un feu de circulation simple :

- **Toutes les transitions** ont lieu sur le **front montant** de l'horloge
- Le bouton 0 et la LED 0 sont à droite de leurs rangées respectives
- Les deux feux ne doivent **jamais** être **verts ou rouges simultanément**, ni **vert pour l'un et orange pour l'autre**. Si un feu est **rouge**, l'autre est forcément **vert ou orange** ; si un feu est **vert**, l'autre est forcément **rouge**. (Il en va de la sécurité des usagers !)
- Dans l'**état initial**, qui est aussi l'état de réinitialisation (état atteint lorsque le **reset** est actionné), les deux feux doivent être **éteints**

- Lorsque le système est dans son **état initial**, chaque bouton **allume le feu correspondant en vert** :
 - Si le bouton 0 est actionné, le feu 0 doit passer directement au vert
 - Si le bouton 1 est actionné, c'est le feu 1 qui doit passer au vert
- Lorsque le système est dans un **état de fonctionnement** (c'est-à-dire, différent de son état initial), les boutons 0 et 1 font office de boutons d'appel piéton. Lorsque le bouton 0 (resp. 1) est actionné,
 - Si le feu 0 (resp. 1) est vert, il doit passer **à l'orange pendant deux secondes, puis au rouge** (indice : l'horloge qui régit le fonctionnement du FPGA est cadencée à 125 MHz)
 - Si le feu 0 (resp. 1) n'est pas vert, l'appui est sans effet
- Le **reset** est **asynchrone** et est relié à un **appui simultané** des deux boutons 0 et 1.¹

Vous êtes libres d'implémenter ces fonctionnalités avec autant d'états que vous le désirez. À titre d'indication, la correction du labo implémente cette spécification avec **5 états** (dont l'état initial).

! Évaluation

Représentez la machine que vous allez implémenter sous forme d'un **diagramme d'états**, que vous incluez dans votre rapport. Le diagramme doit comporter tous les états utilisés, ainsi que toutes les transitions en fonction des entrées, et les valeurs des sorties pour chaque état.

! Évaluation

Cette machine est-elle une machine de Moore, ou une machine de Mealy ? Indiquez-le dans votre rapport, et **justifiez** succinctement votre réponse (une phrase devrait suffire).

Partie 2 : Implémentation en VHDL et simulation

! Attention

L'implémentation de machines à états peut se révéler difficile si vous n'êtes pas bien au fait de la signification de **signal**, **variable**, **process**, et des listes de sensibilité en VHDL : reportez-vous soigneusement aux concepts vus en classe et assurez-vous de les avoir compris avant de vous lancer dans les tests.

¹Rappelez-vous de ce que vous avez implémenté dans la partie 0 et pourquoi !

2a. Implémentation de la machine

Modifiez le fichier `feu.vhd` pour respecter la spécification ci-dessus. Pensez à **bien commenter** votre code pour expliquer son fonctionnement, et à produire un code **clair, correct et bien présenté** (indentation, noms de variables, etc.).

! Évaluation

Faites un *push* de vos modifications pour implémenter la spécification dans `feu.vhd` sur votre dépôt Git.

2b. Vérification par simulation

Pour valider votre module avant de le faire tourner sur la carte, il vous est demandé d'écrire un **banc d'essai** qui permette de tester **toutes ses transitions**. Vous devez modifier le fichier `feu_tb.vhd` pour l'adapter à votre design.

! Évaluation

Simulez votre machine à états à l'aide de votre banc d'essai dans Vivado et incluez des **captures d'écran** des résultats de simulation dans votre rapport. Accompagnez ces captures de quelques **phrases explicatives**, en justifiant par exemple comment elles vous permettent de conclure au bon fonctionnement du module. *N'oubliez pas de push vos modifications régulièrement.*

Partie 3 : Implémentation matérielle

Une fois votre module implémenté et validé, vous pourrez passer à l'implémentation sur la PYNQ (reportez-vous au tutoriel du laboratoire 0 si vous ne vous souvenez pas comment faire). Notez que cette fois-ci, toutes les interactions avec la carte se feront directement avec ses entrées-sorties ; vous n'aurez donc pas besoin d'utiliser de *notebook* Jupyter.

Générez le bitstream. Par défaut, Vivado le met sous `<dossier_projet>/<nom_projet>.runs/impl_1/`. Pour trouver son emplacement exact, regardez la commande `write_bitstream` dans la console TCL.

! Évaluation

Ajoutez votre **bitstream** à votre dépôt, sous le nom **feu_pt1.bit**, dans le dossier `synth_impl`. *N'oubliez pas de renommer le bitstream avant de passer aux sections suivantes, pour éviter de l'écraser lorsque vous générerez d'autres bitstreams.*

Partie 4 : Défi - feu clignotant

Mise en garde. Compléter correctement les parties précédentes peut donner une note de 17 / 20 (85%), ce qui peut normalement être interprété comme un A. La partie 4 demande du travail supplémentaire qui sort normalement des attentes du cours. Il n'est pas nécessaire de la compléter pour réussir le cours ni pour obtenir une bonne note. Il n'est pas recommandé de s'y attaquer si vous éprouvez des difficultés dans un autre cours. La partie 4 propose un défi supplémentaire pour les personnes qui souhaitent s'investir davantage dans le cours INF3500 en toute connaissance de cause.

4a. Ajout de fonctionnalité

Dans un souci de réalisme à toute épreuve, ce défi vous demande d'ajouter une fonctionnalité au feu de circulation que vous avez implémenté : faire clignoter le feu orange avant son passage au rouge. Plus précisément, lorsque le bouton d'appel du feu 0 (resp. 1) est actionné, le feu 0 (resp. 1) doit :

- Passer au **orange fixe pendant une seconde**
- **Clignoter en orange** pendant une seconde supplémentaire

Le délai entre l'appui du bouton et le passage au rouge est toujours de deux secondes, mais le feu doit maintenant clignoter pendant la seconde moitié du délai. Nous vous laissons choisir la fréquence de clignotement du feu, du moment qu'elle est visible (indice : l'horloge est toujours cadencée à 125 MHz).

! Évaluation

Indiquez dans votre rapport la fréquence de clignotement en Hertz du feu orange, choisie et le nombre de ticks d'horloge correspondant, en justifiant.

i Conseil

Vous pouvez implémenter cette fonctionnalité dans un **process à part**. Cela vous assurera de **séparer les fonctionnalités**, et en plus d'être bien plus lisible pour le correcteur, cela vous permettra de **réduire les risques de casser votre module** et de perdre le travail que vous avez accompli dans les parties précédentes. Cependant, vous devrez utiliser les signaux intelligemment, car **VHDL ne permet pas que deux sources différentes agissent sur la même sortie**.

Dans tous les cas, **indiquez clairement en commentaire** quelles parties du code concernent le défi, et quelles parties concernent les parties précédentes. **Vous perdrez des points si ce n'est pas clairement présenté.**

! Évaluation

Faites un *push* de vos modifications dans `feu.vhd`.

4b. Implémentation

Une fois votre fonctionnalité implémentée, testez-la avec le banc d'essai, validez-la, et vérifiez son bon fonctionnement sur la carte.

! Évaluation

N'oubliez pas de *push* vos modifications dans `feu.vhd` et `feu_tb.vhd` pour cette partie.

! Évaluation

Ajoutez votre *bitstream* dans votre dépôt, sous le nom `feu_defi.bit`, sous `synth_impl`.
Pensez à déplacer votre bitstream précédent avant d'en générer un nouveau, pour éviter d'avoir à le re-générer.

Remise

La remise se fait directement sur votre dépôt Git. Faites un *push* régulier de vos modifications, et faites un *push* final avant la date limite de la remise. Respectez l'arborescence de fichiers originale. Consultez le barème de correction pour la liste des fichiers à remettre.

Directives spéciales :

- Ne modifiez pas les noms des fichiers, entités ou architectures, ni les listes de `generics` ou de ports
- Remettez du code de très bonne qualité, lisible, bien indenté et commenté.
- Indiquez clairement la source de tout code que vous réutilisez ou dont vous vous inspirez.
- Remettez votre rapport dans le fichier `rapport.pdf` à la racine du dépôt

Barème de correction

Le barème de correction est progressif. Il est relativement facile d'obtenir une note de passage (> 10) au laboratoire et il faut mettre du travail pour obtenir l'équivalent d'un A (17/20). Obtenir une note plus élevée (jusqu'à 20/20) nécessite plus de travail que ce qui est normalement demandé dans le cadre du cours et plus que les 9 heures que vous devez normalement passer par semaine sur ce cours.

Critères	Points
Partie 0	
• Capture de simulation	0.5
• Pourquoi la simulation fonctionne mais la carte fait n'importe quoi ?	1.5
• Votre process VHDL pour régler le problème de la question précédente	2
Partie 1	
• Votre diagramme d'états et de transitions	1
• Votre machine est-elle une machine de Moore, ou de Mealy ?	1
Partie 2	
• Votre implémentation en VHDL dans <code>feu.vhd</code> (2a.)	4
• Votre banc d'essai complet dans <code>feu_tb.vhd</code> et vos captures d'écran (2b.)	3
Partie 3	
• Votre fichier <code>feu_pt1.bit</code>	2
Général	
• Qualité, lisibilité et élégance du code : indentation, choix des identificateurs, qualité et pertinence des commentaires, respect des consignes de remise incluant les noms des fichiers, orthographe, etc.	2
Sous-total	17
Partie 4	
• Votre justification de la fréquence de clignotement (4a.)	1
• Vos fichiers <code>feu.vhd</code> , <code>feu_tb.vhd</code> et <code>feu_defi.bit</code> (4b.)	2
Total	20